

# MAXX STEELE™

## PROGRAMMER'S TECHNICAL REFERENCE GUIDE



# RoboForce

Version 1.0 | May 24, 2024

# Maxx Steele Technical Manual

Maxx-Steele-1984-Robot contributors

2026-06-24

## Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Maxx Steele Technical Manual</b>               | <b>3</b>  |
| 1.1      | What this guide covers . . . . .                  | 3         |
| 1.2      | How to use this reference guide . . . . .         | 3         |
| 1.3      | Applications guide . . . . .                      | 4         |
| 1.4      | Table of contents . . . . .                       | 4         |
| 1.5      | Firmware images in this archive . . . . .         | 4         |
| <b>2</b> | <b>Chapter 1 — Bytecode Programming Rules</b>     | <b>5</b>  |
| 2.1      | Introduction . . . . .                            | 5         |
| 2.2      | Program storage . . . . .                         | 5         |
| 2.3      | Operating modes (\$0D) . . . . .                  | 5         |
| 2.4      | Status bytes . . . . .                            | 5         |
| 2.5      | Program pointers (zero page) . . . . .            | 6         |
| 2.6      | Programming techniques . . . . .                  | 6         |
| 2.7      | Beginner checklist . . . . .                      | 6         |
| <b>3</b> | <b>Chapter 2 — Opcode Vocabulary</b>              | <b>7</b>  |
| 3.1      | Motion and I/O opcodes (\$00-\$10) . . . . .      | 7         |
| 3.2      | Extended opcodes (\$80-\$FF) . . . . .            | 7         |
| 3.3      | Keypad-mapped opcodes . . . . .                   | 8         |
| 3.4      | Alphabetical index . . . . .                      | 8         |
| 3.5      | Unknown opcodes . . . . .                         | 8         |
| <b>4</b> | <b>Chapter 3 — Programming Motion and Display</b> | <b>9</b>  |
| 4.1      | Motion overview . . . . .                         | 9         |
| 4.2      | Drive opcodes . . . . .                           | 9         |
| 4.3      | Arm and wrist opcodes . . . . .                   | 9         |
| 4.4      | Home opcode \$0B . . . . .                        | 9         |
| 4.5      | Lamp opcode \$0A . . . . .                        | 9         |
| 4.6      | LED segment display . . . . .                     | 10        |
| 4.7      | Programming techniques . . . . .                  | 10        |
| 4.8      | Known gaps . . . . .                              | 10        |
| <b>5</b> | <b>Chapter 4 — Programming Speech and Music</b>   | <b>11</b> |
| 5.1      | Speech overview . . . . .                         | 11        |
| 5.2      | ROM phrases (\$82 / \$0E) . . . . .               | 11        |
| 5.3      | RAM phrases (\$83) . . . . .                      | 11        |
| 5.4      | Music overview . . . . .                          | 12        |

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| 5.5       | Song number opcode \$0D . . . . .                       | 12        |
| 5.6       | Advanced notes . . . . .                                | 12        |
| <b>6</b>  | <b>Chapter 5 — Cartridge Bootstrap and Internal ROM</b> | <b>13</b> |
| 6.1       | What is the internal ROM? . . . . .                     | 13        |
| 6.2       | What is a cartridge? . . . . .                          | 13        |
| 6.3       | Cartridge ROM layout (\$A000 base) . . . . .            | 13        |
| 6.4       | Bootstrap stub (demo cart) . . . . .                    | 13        |
| 6.5       | Key internal ROM addresses . . . . .                    | 14        |
| 6.6       | 6502 vs bytecode — when to use which . . . . .          | 14        |
| 6.7       | Tools and authoring . . . . .                           | 14        |
| 6.8       | Known gaps . . . . .                                    | 14        |
| <b>7</b>  | <b>Chapter 6 — Input/Output Guide</b>                   | <b>15</b> |
| 7.1       | Memory map summary . . . . .                            | 15        |
| 7.2       | Remote keypad (RF input) . . . . .                      | 15        |
| 7.3       | Cartridge expansion . . . . .                           | 16        |
| 7.4       | On-board speech and face hardware . . . . .             | 16        |
| 7.5       | CPU and support chips . . . . .                         | 16        |
| 7.6       | Schematics . . . . .                                    | 16        |
| <b>8</b>  | <b>Appendices</b>                                       | <b>17</b> |
| 8.1       | A — Opcode abbreviations . . . . .                      | 17        |
| 8.2       | B — Display segment table . . . . .                     | 17        |
| 8.3       | C — Memory map . . . . .                                | 17        |
| 8.4       | D — Zero-page registers . . . . .                       | 17        |
| 8.5       | E — Status bits . . . . .                               | 18        |
| 8.6       | F — Music duration table . . . . .                      | 18        |
| 8.7       | G — Cartridge file layout . . . . .                     | 18        |
| 8.8       | H — maxx_rom.py commands . . . . .                      | 18        |
| 8.9       | I — Internal ROM entry points . . . . .                 | 18        |
| 8.10      | J — Messages and errors . . . . .                       | 19        |
| 8.11      | K — Bibliography . . . . .                              | 19        |
| 8.12      | L — Glossary . . . . .                                  | 19        |
| <b>9</b>  | <b>Maxx Steele Quick Reference Card</b>                 | <b>20</b> |
| 9.1       | Modes (\$0D) . . . . .                                  | 20        |
| 9.2       | Key zero page . . . . .                                 | 20        |
| 9.3       | Motion opcodes . . . . .                                | 20        |
| 9.4       | Speech / music . . . . .                                | 20        |
| 9.5       | Cartridge header (\$A000) . . . . .                     | 21        |
| 9.6       | Common sequences . . . . .                              | 21        |
| 9.7       | Tools . . . . .                                         | 21        |
| <b>10</b> | <b>Schematic Diagram Index</b>                          | <b>22</b> |
| 10.1      | Main logic board . . . . .                              | 22        |
| 10.2      | Transmitter (remote) . . . . .                          | 22        |
| 10.3      | Receiver (robot RF) . . . . .                           | 22        |
| 10.4      | Cartridge . . . . .                                     | 22        |
| 10.5      | Face / speech / display . . . . .                       | 23        |
| 10.6      | Power module . . . . .                                  | 23        |

|                                       |    |
|---------------------------------------|----|
| 10.7Paddle mirror accessory . . . . . | 23 |
| 10.8Chip cross-reference . . . . .    | 23 |

## 1 Maxx Steele Technical Manual

This manual describes how to program the 1984 CBS Toys / Ideal **Maxx Steele** robot at the software level: bytecode motion programs, speech, music, cartridge images, and the 6502 internal ROM that interprets them.

It is modeled on the structure of the Commodore 64 Programmer’s Reference Guide — numbered chapters, appendices, a quick-reference card, and a schematic index — adapted to Maxx Steele hardware.

**PDF (latest):** `Maxx-Steele-Technical-Manual.pdf` — front/rear covers plus chapters; rebuilt when `.md` or cover images change (`python3 tools/build_technical_manual_pdf.py` locally; GitHub Actions on push to main).

### 1.1 What this guide covers

| Document                                               | Audience                       | Focus                                                        |
|--------------------------------------------------------|--------------------------------|--------------------------------------------------------------|
| <b>This guide</b>                                      | Programmers, reverse engineers | Bytecode language, memory map, I/O, internal ROM hooks       |
| Cartridge/PROGRAMMING.md<br>(Cartridge/PROGRAMMING.md) | Cartridge authors              | Step-by-step cart layout, tools, demo walkthrough            |
| Chassis/Manual/<br>(Chassis/Manual/)                   | Owners                         | Factory user / reference manuals (operation, not ROM layout) |

Primary sources: `R. Wind disassemblies (maxx_internal_ROM.dsm (Chassis/Firmware/Assembly/maxx_demo_ROM_532.dsm (Cartridge/Firmware/Assembly/maxx_demo_ROM_532.dsm)), tools/maxx_rom.py)`.

### 1.2 How to use this reference guide

**If you are new to Maxx programming**, read in order:

1. Chapter 1 — Bytecode programming rules
2. Chapter 2 — Opcode vocabulary
3. Chapter 3 — Motion and display
4. Work through the demo program in `Cartridge/PROGRAMMING.md (Cartridge/PROGRAMMING.md)` §5

**If you are writing cartridges**, add:

5. Chapter 5 — Cartridge bootstrap and internal ROM
6. `Cartridge/PROGRAMMING.md (Cartridge/PROGRAMMING.md)` §9 (authoring workflow)

**If you are reverse engineering hardware**, use:

- Chapter 6 — Input/output guide
- Schematics
- Annotated `.dsm` listings under `Chassis/Firmware/ (Chassis/Firmware/)` and `Cartridge/Firmware/ (Cartridge/Firmware/)`

Keep Quick reference and Appendices open while coding.

## 1.3 Applications guide

| Application                | Mode (\$00) | What you need                                           |
|----------------------------|-------------|---------------------------------------------------------|
| Remote control (immediate) | 0           | Remote keypad; opcodes executed live                    |
| Learn key sequences        | 1           | Records into program RAM                                |
| Enter program steps        | 2           | Keypad maps to opcodes via \$E6B5 table                 |
| Run stored program         | 3           | Bytecode at \$0200, terminator FF FF                    |
| Factory demo               | 3 + cart    | MAXXCART.532 (Cartridge/Firmware/Binary/MAXXCART.532)   |
| Custom cartridge           | 3 + cart    | bootstrap 4 KB EPROM image; see Ch 5 + cartridge manual |
| Speech / music authoring   | any         | Ch 4; phrase tables in cart or RAM                      |
| Repair / RE                | —           | Ch 6, Schematics, DataSheets/ (DataSheets/)             |

## 1.4 Table of contents

| Chapter | Title                                |
|---------|--------------------------------------|
| —       | Introduction (this file)             |
| 1       | Bytecode programming rules           |
| 2       | Opcode vocabulary                    |
| 3       | Programming motion and display       |
| 4       | Programming speech and music         |
| 5       | Cartridge bootstrap and internal ROM |
| 6       | Input/output guide                   |
| A-L     | Appendices                           |
| —       | Quick reference card                 |
| —       | Schematic diagram index              |

## 1.5 Firmware images in this archive

| Image                 | Path                                                                            |
|-----------------------|---------------------------------------------------------------------------------|
| Internal 8 KB ROM     | Chassis/Firmware/Binary/Maxxrom.64 (Chassis/Firmware/Binary/Maxxrom.64)         |
| Demo cartridge (4 KB) | Cartridge/Firmware/Binary/MAXXCART.532 (Cartridge/Firmware/Binary/MAXXCART.532) |

Tools: tools/maxx\_rom.py (tools/maxx\_rom.py) — disassemble, validate, template, opcode export.

## 2 Chapter 1 — Bytecode Programming Rules

This chapter describes the rules for writing Maxx Steele **programs**: how bytecode is stored, how operating modes affect entry, and practical techniques for building sequences.

The robot does not run arbitrary user 6502 code for motion demos. The internal ROM interprets a **bytecode program** in RAM. Cartridges supply tables and a short bootstrap stub; see Chapter 5.

### 2.1 Introduction

A Maxx program is a linear sequence of (**opcode, operand**) byte pairs. Each pair is one step. The executor reads from program RAM, displays the opcode name on the LED segment display, and performs the action.

Programs are edited through the remote keypad (Program mode), loaded from a cartridge bootstrap, or built offline and burned into EPROM.

### 2.2 Program storage

| Property      | Value                                                             |
|---------------|-------------------------------------------------------------------|
| RAM region    | \$0200-03FE                                                       |
| Step size     | 2 bytes (opcode, operand)                                         |
| Maximum steps | 255 pairs (512 bytes); <b>FULL</b> displayed when \$037F exceeded |
| Terminator    | FF FF (required)                                                  |

Example minimal program:

```
83 10    ; speak RAM phrase 0
0C 03    ; delay 3 seconds
01 10    ; drive forward
0B 00    ; home all joints
FF FF    ; end
```

### 2.3 Operating modes (\$00)

The mode byte in zero page selects how keypad input and the executor behave.

| Value | Name           | Behavior                               |
|-------|----------------|----------------------------------------|
| \$00  | Immediate      | Keys execute motion/speech instantly   |
| \$01  | Learn          | Records key sequences into program RAM |
| \$02  | Program        | Enter bytecode steps via keypad        |
| \$03  | Execute / Game | Run the stored program                 |

Cartridge entry stubs typically initialize status bytes \$02 and \$03 before jumping to the internal main loop at \$E0B6.

### 2.4 Status bytes

#### 2.4.1 \$02 — Status A

Set by cartridge bootstrap (demo uses LDA #\$02 / STA \$02). Bits include speech error (**Ser**), enable backoff (**Ebof**), power-down, drive, and speech flags. Keypad rows toggle subsets; see Appendix E.

### 2.4.2 \$03 — Status B

Demo entry uses LDA #\$82 / STA \$03 (**PDon**, **Edof**, **SPof**, **UCon**). Governs user control and execute gating.

## 2.5 Program pointers (zero page)

| Addr        | Role                                                  |
|-------------|-------------------------------------------------------|
| \$0F / \$10 | Current step in \$0200 table (program mode / execute) |
| \$11 / \$13 | Current opcode and operand during entry               |
| \$24 / \$25 | Program byte pointer used by executor                 |

## 2.6 Programming techniques

### 2.6.1 Delays

Opcode \$0C operand is delay time in **seconds** (decimal). Chain delays between motion commands so mechanics can finish.

0C 02 ; wait 2 seconds

01 14 ; forward (operand scale empirical – see Ch 3)

0C 04 ; wait 4 seconds

### 2.6.2 Homing

Opcode \$0B with operand **\$00 only** — returns joints to a known position before the next move.

### 2.6.3 Speech then motion

RAM phrases (\$83) must be copied to \$0500 before execute (cartridge stub or prior setup). ROM phrases (\$82) need no RAM table.

### 2.6.4 Program length

If program entry exceeds available RAM, the display shows **FULL** (internal ROM compares against \$037F).

### 2.6.5 Marker opcode \$FE

Display-only **beg** marker; does not affect execution flow in the demo cart.

## 2.7 Beginner checklist

1. Every program ends with FF FF.
2. Operands are one byte (\$00-FF); meaning depends on opcode (Ch 2-4).
3. Test with python3 tools/maxx\_rom.py validate before burning EPROM.
4. For a worked 38-step demo, see Cartridge/PROGRAMMING.md (Cartridge/PROGRAMMING.md) §5.

**Next:** Chapter 2 — Opcode vocabulary

## 3 Chapter 2 — Opcode Vocabulary

This chapter lists every known Maxx Steele program opcode. Display names come from the internal ROM segment table at **\$F878** (see Appendix B).

Opcodes are described in **hex order** within each group. For a one-page summary, see Quick reference.

Export machine-readable opcode JSON:

```
python3 tools/maxx_rom.py opcodes Cartridge/opcodes.json
```

### 3.1 Motion and I/O opcodes (\$00-\$10)

| Opcode | Display | Name                   | Operand               |
|--------|---------|------------------------|-----------------------|
| \$00   | L       | Turn / drive left      | Distance or angle     |
| \$01   | F       | Drive forward          | Distance              |
| \$02   | b       | Drive reverse          | Distance              |
| \$03   | r       | Turn / drive right     | Angle                 |
| \$04   | Uu      | Wrist up               | Value                 |
| \$05   | Ud      | Wrist down             | Value                 |
| \$06   | Au      | Arms up                | Value                 |
| \$07   | Ad      | Arms down              | Value                 |
| \$08   | Cr      | Claw rotate            | Value                 |
| \$09   | Cc      | Claw open/close        | \$00=open, \$01=close |
| \$0A   | HL      | Lamp                   | \$00=off, \$01=on     |
| \$0B   | init    | Home (all joints)      | Must be \$00          |
| \$0C   | d       | Delay                  | Seconds               |
| \$0D   | Sn      | Song number reference  | Index                 |
| \$0E   | S       | Speech (ROM phrase)    | Phrase #              |
| \$0F   | SS      | Speech (shift mode)    | Phrase #              |
| \$10   | PS      | Program speech capture | Slot #                |

#### Examples

```
01 14    ; forward
0A 01    ; lamp on
0C 0A    ; delay 10 seconds
82 3F    ; speak ROM phrase $3F
```

### 3.2 Extended opcodes (\$80-\$FF)

Opcodes with bit 7 set (\$80+) are remapped through the display table for execution. The executor treats \$80 as an alias mapping to display index \$0C.

| Opcode | Display | Name                     | Operand                   |
|--------|---------|--------------------------|---------------------------|
| \$81   | PLAY    | Play tune from music RAM | Tune # (\$0400 table)     |
| \$82   | SPEE    | Speak ROM phrase         | Phrase #                  |
| \$83   | SS      | Speak RAM phrase         | Phrase # in \$0500        |
| \$84   | CLr     | Clear speech phrase slot | Slot #                    |
| \$FE   | beg     | Program begin marker     | Display only              |
| \$FF   | End     | End of program           | Must be \$FF (pair FF FF) |



### 3.3 Keypad-mapped opcodes

The internal ROM maps remote key scan codes to opcodes via table **\$E6B5**. Faceplate names: Transmitter/remote-keypad.md (Transmitter/remote-keypad.md). Documented mapping (from maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/maxx\_internal\_ROM.dsm)):

Key opcodes: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  
41 46 43 13 84 82 81 83 80

| Opcode    | Typical keypad use                    |
|-----------|---------------------------------------|
| \$41      | Game-related step                     |
| \$43      | Click                                 |
| \$46      | Motion step                           |
| \$13      | (mapped key — see learn/program mode) |
| \$80-\$84 | Extended speech/music/clear group     |

In **Program mode**, pressing a key stores the mapped opcode and prompts for an operand byte.

### 3.4 Alphabetical index

| Name            | Opcode |
|-----------------|--------|
| arms_down       | \$07   |
| arms_up         | \$06   |
| claw_open_close | \$09   |
| claw_rotate     | \$08   |
| delay           | \$0C   |
| drive_forward   | \$01   |
| drive_reverse   | \$02   |
| end             | \$FF   |
| home            | \$0B   |
| lamp            | \$0A   |
| play_tune       | \$81   |
| song_number     | \$0D   |
| speak_ram       | \$83   |
| speak_rom       | \$82   |
| speech_clear    | \$84   |
| speech_program  | \$10   |
| speech_rom      | \$0E   |
| speech_shift    | \$0F   |
| turn_left       | \$00   |
| turn_right      | \$03   |
| wrist_down      | \$05   |
| wrist_up        | \$04   |

### 3.5 Unknown opcodes

If the executor encounters an undefined opcode, behavior is undefined in this documentation. Stick to opcodes listed above and validated with tools/maxx\_rom.py (tools/maxx\_rom.py).

**Previous:** Chapter 1 · **Next:** Chapter 3 — Motion and display

## 4 Chapter 3 — Programming Motion and Display

This chapter describes motion opcodes, the headlamp, and how the LED display shows program steps during entry and execution.

Distance and angle **operands are empirical** — the internal ROM scales them to motor timing. Values in the factory demo (e.g. \$14 forward, \$28 arm raise) are starting points, not documented engineering units.

### 4.1 Motion overview

Drive and joint motion opcodes \$00-\$09 write timing values to motor control hardware mapped around **\$1600** and **\$1C00** (see Chapter 6).

The executor runs each step to completion (or until interrupted) before advancing the program pointer at \$0200.

### 4.2 Drive opcodes

| Opcode | Action     | Demo cart examples |
|--------|------------|--------------------|
| \$00   | Turn left  | 00 05              |
| \$01   | Forward    | 01 14, 01 10       |
| \$02   | Reverse    | 02 14              |
| \$03   | Turn right | 03 06              |

Operands control how long or how far the drive system runs. Compare steps in `maxx_demo_ROM_532.dsm` (Cartridge/Firmware/Assembly/`maxx_demo_ROM_532.dsm`) with on-robot behavior when calibrating new values.

### 4.3 Arm and wrist opcodes

| Opcode | Action          | Demo example |
|--------|-----------------|--------------|
| \$04   | Wrist up        | —            |
| \$05   | Wrist down      | 05 23        |
| \$06   | Arms up         | 06 28        |
| \$07   | Arms down       | —            |
| \$08   | Claw rotate     | 08 15        |
| \$09   | Claw open/close | 09 00 (open) |

### 4.4 Home opcode \$0B

Always use operand **00**:

0B 00 ; home – all joints to reference position

The demo cart homes before backing up at the end of the sequence.

### 4.5 Lamp opcode \$0A

| Operand | Effect   |
|---------|----------|
| \$00    | Lamp off |
| \$01    | Lamp on  |

Demo sequence: 0A 01 ... delay ... 0A 00.

## 4.6 LED segment display

During program entry and execution, the internal ROM fetches a **two-byte display code** from table **\$F878** indexed by the current opcode (see LDA \$F878,Y in maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/maxx\_internal\_ROM.dsm)). This drives the shift-register display at **\$1200**.

Display names in Chapter 2 match that table (e.g. **L**, **F**, **PLAY**, **d**).

When program RAM is full, the display spells **FULL** (bytes at internal ROM label referencing \$037F limit).

## 4.7 Programming techniques

1. **Home after motion blocks** — reduces accumulated joint error.
2. **Delay after every drive** — allows the chassis to settle before the next command.
3. **Lamp as status** — signal “busy” during long speech delays.
4. **Small operand sweeps** — test \$01 08, \$01 10, \$01 14 on your robot and record what works.

## 4.8 Known gaps

- Exact mm/degree mapping for operands is not documented in the internal ROM listing.
- Motor ramp profiles and limit switches are hardware-dependent; see Mainboard/Schematic/ (Mainboard/Schematic/) and future KiCad RE.

**Previous:** Chapter 2 · **Next:** Chapter 4 — Speech and music

## 5 Chapter 4 — Programming Speech and Music

This chapter describes how the Maxx Steele robot produces speech and music under program control.

Speech uses a parallel phoneme interface at **\$1400**. The speech driver subroutine **\$F3D5** clocks 4-bit nybbles to the synthesizer. Music uses byte pairs in RAM at **\$0400** and an IRQ handler near **\$F1D8**.

### 5.1 Speech overview

| Mechanism              | Opcode     | Data source                                |
|------------------------|------------|--------------------------------------------|
| ROM phrase table       | \$82, \$0E | Internal ROM phoneme strings               |
| RAM phrase table       | \$83       | \$0500 (copied from cartridge or recorded) |
| Program speech capture | \$10       | User-recorded slots                        |
| Clear phrase slot      | \$84       | Clears RAM slot                            |

### 5.2 ROM phrases (\$82 / \$0E)

Phrases are indexed phoneme strings in internal ROM. The driver at **\$F3D5** outputs sounds with index **X** (see JSR **\$F3D5** in `maxx_internal_ROM.dsm` (Chassis/Firmware/Assembly/`maxx_internal_ROM.dsm`)). Phoneme code tables reside at **\$F567** / **\$F4DB** (140 entries). Until the full **\$F4DB** table is transcribed, treat phrase indices as opaque IDs.

Demo examples:

```
82 3F      ; "Ha ha ha ha ha" (ROM)
83 16      ; uses ROM index via RAM phrase indirection in demo – see cart listing
```

### 5.3 RAM phrases (\$83)

Cartridges embed phrase data at offset **\$81** (address **\$A081** for demo cart). The bootstrap copies bytes to **\$0500**.

Each phrase is a sequence of **16-bit phoneme tokens**; **\$FF** pads unused slots.

Demo cart phrase table (from `maxx_demo_ROM_532.dsm` (Cartridge/Firmware/Assembly/`maxx_demo_ROM_532.dsm`))

| Slot | Content (abbrev.)                           |
|------|---------------------------------------------|
| 0    | “Hello, I am Maxx Steele” / greeting tokens |
| 1    | “I am ready when you are”                   |
| 2    | “I am a great match for humans”             |
| 3    | “Goodbye for now, have a good day”          |

Example program bytes:

```
83 10      ; speak RAM phrase 0 (demo uses operand $10 for table indexing –
verify in .dsm)
0C 02      ; delay
83 00      ; another RAM phrase
```

**Note:** Demo listing uses operands \$10, \$00, \$01, etc. — cross-check against your cart image with `maxx_rom.py disasm`.

## 5.4 Music overview

Opcode **\$81** plays a tune from the music RAM table at **\$0400**.

Tune selection calls **\$EF01** to resolve pointers for tune number in the operand (multiple call sites in internal ROM).

### 5.4.1 Music byte pairs

Stored in **\$0400** (and cartridge offset **\$BB / \$A0BB**):

```
70 12    ; note
70 11    ; note
38 0F
54 0D
E0 0F
00 00    ; end of tune
```

The music IRQ handler at **\$F1D8** reads duration data from **\$F15B** and frequency/note data from **\$0400**.

Demo: 81 06 plays tune #6 (Reveille); 81 00 plays tune #0 at end of demo sequence.

## 5.5 Song number opcode \$0D

References a song index for the internal song table (distinct from \$81 play-tune in some keypad contexts). Used in learn/program flows.

## 5.6 Advanced notes

- **Phoneme authoring** requires transcribing **\$F4DB / \$F567** — listed as future work in Cartridge/PROGRAMMING.md (Cartridge/PROGRAMMING.md) §10.
- **Speech error flag (SEr** in **\$02**) — set when speech subsystem reports fault; demo cart entry sets related status.
- Face/speech hardware datasheets: DataSheets/ (DataSheets/) (Sanyo LC3100/LC8100, ET9420 family).

**Previous:** Chapter 3 · **Next:** Chapter 5 — Cartridge and internal ROM

## 6 Chapter 5 — Cartridge Bootstrap and Internal ROM

This chapter explains the relationship between **6502 machine code** in cartridge ROM and the **bytecode interpreter** in the robot's 8 KB internal ROM.

### 6.1 What is the internal ROM?

At power-on the **MOS 6502** CPU executes firmware in **\$E000-FFFF** (8 KB). This code is the robot's only native language — analogous to the C64 KERNAL + BASIC ROM, but specialized for:

- Keypad scan and mode handling
- Bytecode fetch/decode from \$0200
- Speech, music, motor, and display drivers
- Cartridge detection and bootstrap handoff

User programs are **not** generally written in 6502. They are bytecode tables interpreted by this ROM.

Source listing: Chassis/Firmware/Assembly/maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/Binary: Chassis/Firmware/Binary/Maxxrom.64 (Chassis/Firmware/Binary/Maxxrom.64)

### 6.2 What is a cartridge?

A **4 KB EPROM** plugs into the cartridge slot, mapped into **\$2000-B000** in 4 KB steps. The factory demo sits at **\$A000**.

A valid image supplies:

1. Entry vector and copyright header
2. A short **6502 bootstrap stub**
3. Program, speech, and music **data tables**

The stub copies tables into RAM and jumps to the internal main loop — it does **not** replace the interpreter.

### 6.3 Cartridge ROM layout (\$A000 base)

| Offset | Size  | Content                          |
|--------|-------|----------------------------------|
| \$00   | 2     | Entry vector (word), e.g. \$A013 |
| \$02   | 17    | Copyright: (c) 1985 CBS Toys     |
| \$13   | code  | Bootstrap stub (6502)            |
| \$35   | table | Program bytecode                 |
| \$81   | table | Speech phrases (opcode \$83)     |
| \$BB   | table | Music notes (opcode \$81)        |

Cartridge detection scans 4 KB boundaries from \$2000 upward, comparing the copyright string at offset +2.

### 6.4 Bootstrap stub (demo cart)

Entry at **\$A013** (maxx\_demo\_ROM\_532.dsm (Cartridge/Firmware/Assembly/maxx\_demo\_ROM\_532.dsm))

1. STA \$02 / STA \$03 — initialize status flags
2. Copy program from **\$A035 → \$0200**
3. Copy phrases from **\$A081 → \$0500**
4. Copy music from **\$A0BB → \$0400**
5. **JMP \$E0B6** — enter internal ROM main loop

Expected entry prologue: A9 02 85 02 (LDA #\$02 / STA \$02). tools/maxx\_rom.py validate (tools/maxx\_rom.py) checks this.

## 6.5 Key internal ROM addresses

| Address | Role                                             |
|---------|--------------------------------------------------|
| \$E0B6  | Main loop entry after cartridge bootstrap        |
| \$E0D1  | Alternate loop / mode handler (JMP \$E0D1 sites) |
| \$E6B5  | Keycode → opcode translation table               |
| \$F878  | Opcode → LED segment display table               |
| \$F3D5  | Speech output subroutine                         |
| \$F1D8  | Music IRQ handler region                         |
| \$EF01  | Tune pointer resolver (music)                    |
| \$F15B  | Note duration table                              |

## 6.6 6502 vs bytecode — when to use which

| Task                         | Language                                        |
|------------------------------|-------------------------------------------------|
| Motion/speech demo sequence  | Bytecode in \$0200                              |
| Custom phrases/music in cart | Data tables + bootstrap                         |
| Patch OS behavior            | 6502 patch (advanced RE only)                   |
| New cartridge program        | Bootstrap stub (minimal 6502) + bytecode tables |

Do not place arbitrary 6502 execution paths in the demo program area unless you fully understand cartridge mapping and internal ROM expectations.

## 6.7 Tools and authoring

Step-by-step cartridge creation: Cartridge/PROGRAMMING.md (Cartridge/PROGRAMMING.md) §9.

```
python3 tools/maxx_rom.py template mycart.532
python3 tools/maxx_rom.py validate mycart.532
python3 tools/maxx_rom.py disasm mycart.532 --compare-dsm Cartridge/Firmware/Assembly/
```

EPROM type: Mitsubishi KM2365 family — DataSheets/Mitsubishi-KM2365.pdf (DataSheets/Mitsubishi-KM2365.pdf).

## 6.8 Known gaps

- Full subroutine catalog for \$E000–FFFF not yet indexed (see Appendix I).
- Padding at \$A0C7+ in demo image marked “not Maxx-related” in .dsm — treat as unused.

**Previous:** Chapter 4 · **Next:** Chapter 6 — I/O guide

# 7 Chapter 6 — Input/Output Guide

This chapter describes memory-mapped hardware, the remote keypad, cartridge slot, and the 27 MHz radio control link.

## 7.1 Memory map summary

| Region        | Address        | Purpose                       |
|---------------|----------------|-------------------------------|
| Zero page     | \$0000–\$00FF  | Flags, pointers, keypad state |
| Warm start    | \$0100–\$0103  | Copyright mirror              |
| Stack         | \$01FF ↓       | 6502 stack                    |
| Program RAM   | \$0200–\$03FE  | Bytecode (2 bytes/step)       |
| Music RAM     | \$0400–\$04FF  | Note pairs                    |
| Speech RAM    | \$0500+        | Custom phrases                |
| Timer / misc  | \$1000         | I/O                           |
| Display       | \$1200         | Shift-register LEDs           |
| Speech chip   | \$1400         | Parallel phoneme output       |
| Motors        | \$1600, \$1C00 | Drive and joint timing        |
| Cartridge ROM | \$2000–\$B000  | 4 KB slot stepping            |
| Internal ROM  | \$E000–\$FFFF  | 8 KB operating system         |

Full table: Appendix C.

## 7.2 Remote keypad (RF input)

The handheld transmitter uses a **COP411L** MCU scanning a row/column matrix. Key events are encoded as **27 MHz OOK** packets received by the robot superhet strip.

### 7.2.1 Matrix layout

|     | L7 | L6 | L5 | L4 | PK |
|-----|----|----|----|----|----|
| D0  | A  | B  | C  | D  |    |
| D1  | E  | F  | G  | H  |    |
| L0  | I  | J  | K  | L  |    |
| L1  | M  | N  | O  | P  |    |
| L2  | Q  | R  | S  | T  |    |
| L3  | U  | V  | W  | X  |    |
| Gnd |    |    |    |    | Y  |

Faceplate labels (from Remote-Front.jpg (Transmitter/Photos/Product/Remote-Front.jpg)): **U/1/2/3 DRIVE, 4/5 WRIST, 6/7 ARMS, 8/9 CLAW, A LAMP, B HOME, NOTE REST (WAIT), SHIFT OCTAVE, CLEAR, ENTER, SONG/NOTES, CLOCK/STATUS, SPEECH, MOTION, GAME, PROGRAM, LEARN, EXECUTE, POWER/STOP (Y).**

Full key-to-matrix table: Transmitter/remote-keypad.md (Transmitter/remote-keypad.md).  
Diagram: Remote-Front.svg (Transmitter/Photos/Product/Remote-Front.svg).

Matrix wiring refs: keyboard-matrix-reference-1.png (Transmitter/Photos/ReverseEngineering/matrix-reference-1.png), keyboard-matrix-reference-2.png (Transmitter/Photos/ReverseEngineering/matrix-reference-2.png).



### 7.2.2 RF link (summary)

- Carrier: **27 MHz** on-off keying
- MCU clock: **455 kHz** ceramic resonator (IF reference)
- Bit period: **~1.55 ms**; packet repeat **~29 ms** (Power/Stop **~21 ms**)

Details: Transmitter/transmitter-architecture.md (Transmitter/transmitter-architecture.m  
tools/rfcap/ (tools/rfcap/).

## 7.3 Cartridge expansion

| Property        | Value                                    |
|-----------------|------------------------------------------|
| ROM size        | 4 KB per image                           |
| Mapping         | \$2000, \$6000, \$A000, ... (4 KB steps) |
| Demo cart base  | \$A000                                   |
| Required header | 17-byte CBS Toys copyright at offset +2  |

Hardware EPROM: see DataSheets/Mitsubishi-KM2365.pdf (DataSheets/Mitsubishi-KM2365.pdf).

## 7.4 On-board speech and face hardware

Speech synthesis ICs documented in DataSheets/ (DataSheets/) include Sanyo LC3100/LC8100 and Eurotechnique ET9420 family parts. The CPU writes phoneme nybbles to **\$1400**.

Face/display COP41xL subsystem: National-COP41xL-Display-Motors.pdf (DataSheets/National-COP41xL-Display-Motors.pdf) (refdes **U500** in archive naming).

## 7.5 CPU and support chips

| Part          | Role          | Datasheet                                                 |
|---------------|---------------|-----------------------------------------------------------|
| MOS 6502      | Main CPU      | MOS-6502.pdf<br>(DataSheets/MOS-6502.pdf)                 |
| COP420 family | Auxiliary MCU | National-COP420.pdf<br>(DataSheets/National-COP420.pdf)   |
| M6116         | Static RAM    | Mitsubishi-M6116.pdf<br>(DataSheets/Mitsubishi-M6116.pdf) |

CPU pinout notes (project doc): Maxx-Steele-CPU-Pinout.pdf (DataSheets/Maxx-Steele-CPU-Pinout.pdf).

## 7.6 Schematics

See Schematic diagram index for board-level drawings.

**Previous:** Chapter 5 · **Next:** Appendices

## 8 Appendices

Reference tables for the Maxx Steele Programmer's Reference Guide.

### 8.1 A — Opcode abbreviations

See Quick reference for the full hex table. Names match tools/maxx\_rom.py (tools/maxx\_rom.py) OPCODES dict.

### 8.2 B — Display segment table

Internal ROM table **\$F878** maps opcode indices to two-character LED segment patterns. Documented display names:

| Index | Display                | Opcode context    |
|-------|------------------------|-------------------|
| —     | L, F, b, r             | Drive             |
| —     | Uu, Ud, Au, Ad, Cr, Cc | Joints            |
| —     | HL, init, d            | Lamp, home, delay |
| —     | PLAY, SPEE, SS, CLr    | Extended          |
| —     | End, beg               | \$FF, \$FE        |

Full glyph bitmaps are embedded in maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/maxx\_i near label FULL / \$F878 references.

### 8.3 C — Memory map

| Region       | Address        | Size     | Purpose               |
|--------------|----------------|----------|-----------------------|
| Zero page    | \$0000–\$00FF  | 256 B    | Flags, pointers       |
| Warm start   | \$0100–\$0103  | 4 B      | Copyright mirror      |
| Stack        | \$01FF ↓       | —        | 6502 stack            |
| Program RAM  | \$0200–\$03FE  | 510 B    | Bytecode              |
| Music RAM    | \$0400–\$04FF  | 256 B    | Note pairs            |
| Speech RAM   | \$0500+        | —        | Phrases for \$83      |
| I/O          | \$1000         | —        | Timer                 |
| Display      | \$1200         | —        | LED shift register    |
| Speech       | \$1400         | —        | Phoneme parallel port |
| Motors       | \$1600, \$1C00 | —        | Motor drive           |
| Cartridge    | \$2000–\$B000  | 4K slots | Plug-in ROM           |
| Internal ROM | \$E000–\$FFFF  | 8 KB     | OS / interpreter      |

### 8.4 D — Zero-page registers

| Addr      | Name                 | Purpose                                    |
|-----------|----------------------|--------------------------------------------|
| \$02      | Status A             | Speech error, backoff, power, drive flags  |
| \$03      | Status B             | User control, execute gating               |
| \$0D      | Mode                 | 0=immediate, 1=learn, 2=program, 3=execute |
| \$0F/\$10 | Program pointer      | Step in \$0200                             |
| \$11/\$13 | Current op/operand   | Entry state                                |
| \$24/\$25 | Program byte pointer | Executor                                   |

## 8.5 E — Status bits

### 8.5.1 \$02 (from maxx\_rom.py)

| Bit | Label                   |
|-----|-------------------------|
| 0   | SER (speech error)      |
| 1   | Ebof (enable backoff)   |
| 2   | PDon (power down on)    |
| 3   | Edof (enable drive off) |
| 4   | SPon (speech on)        |

Keypad toggles (internal ROM): row 0 → SER, PAR; row 2 → Edon, Edof; row 3 → UCon, UCof; row 4 → Ebon, Ebof; row 5 → SPon, SPof.

### 8.5.2 \$03

| Bit | Label                  |
|-----|------------------------|
| 0   | UCon (user control on) |
| 1   | SPon (speech enable)   |

Demo bootstrap: \$02 ← \$02, \$03 ← \$82.

## 8.6 F — Music duration table

Note durations for the music IRQ path are read from **\$F15B** (indexed 1-8 in maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/maxx\_internal\_ROM.dsm)). Frequency/note bytes live in **\$0400**.

Complete note name → byte mapping is not yet transcribed in this archive.

## 8.7 G — Cartridge file layout

File offset = cartridge address – \$A000 for demo cart base.

| File offset | Addr   | Content              |
|-------------|--------|----------------------|
| \$0000      | \$A000 | Entry vector         |
| \$0002      | \$A002 | Copyright (17 bytes) |
| \$0013      | \$A013 | Bootstrap code       |
| \$0035      | \$A035 | Program table        |
| \$0081      | \$A081 | Phrase table         |
| \$00BB      | \$A0BB | Music table          |

## 8.8 H — maxx\_rom.py commands

```
python3 tools/maxx_rom.py disasm PATH [--compare-dsm DSM]
python3 tools/maxx_rom.py validate PATH
python3 tools/maxx_rom.py template OUTPUT.532
python3 tools/maxx_rom.py opcodes OUTPUT.json
```

## 8.9 I — Internal ROM entry points

Partial list from maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/maxx\_internal\_ROM.dsm):

| Address | Role                      |
|---------|---------------------------|
| \$E0B6  | Post-bootstrap main loop  |
| \$E0D1  | Mode / executor loop      |
| \$E6B5  | Keycode table base        |
| \$EF01  | Music tune pointer helper |
| \$F1D8  | Music IRQ region          |
| \$F3D5  | Speech output             |
| \$F878  | Display pattern table     |

## 8.10 J — Messages and errors

| Message / flag              | Cause                              |
|-----------------------------|------------------------------------|
| <b>FULL</b>                 | Program exceeds \$037F / RAM limit |
| <b>Ser</b>                  | Speech error status bit            |
| validate copyright mismatch | Cart missing (c) 1985 CBS Toys     |
| Missing FF FF               | Program table not terminated       |

## 8.11 K — Bibliography

- R. Wind — maxx\_internal\_ROM.dsm (Chassis/Firmware/Assembly/maxx\_internal\_ROM.dsm), maxx\_demo\_ROM\_532.dsm (Cartridge/Firmware/Assembly/maxx\_demo\_ROM\_532.dsm) (2002–2006)
- Factory manual — Chassis/Manual/MaxxSteeleReferenceGuide.pdf (Chassis/Manual/MaxxSteele)
- This repository — <https://github.com/webaugur/Maxx-Steele-1984-Robot>
- C64 Programmer’s Reference Guide — structural inspiration (Commodore, 1982)

## 8.12 L — Glossary

| Term      | Meaning                                                  |
|-----------|----------------------------------------------------------|
| Bytecode  | Opcode/operand pairs interpreted by internal ROM         |
| Bootstrap | Short 6502 stub in cartridge that loads RAM tables       |
| OOK       | On-off keying RF modulation (27 MHz)                     |
| IF        | Intermediate frequency (455 kHz in transmitter/receiver) |
| Phoneme   | Speech sound unit; 4-bit codes to \$1400                 |
| Refdes    | Schematic reference designator (U1, U400, ...)           |
| .532      | 4 KB cartridge image file extension used in archive      |

## 9 Maxx Steele Quick Reference Card

Single-page summary. Full detail in chapters 1-6 and Appendices.

### 9.1 Modes (\$00)

| Val | Mode      |
|-----|-----------|
| 0   | Immediate |
| 1   | Learn     |
| 2   | Program   |
| 3   | Execute   |

### 9.2 Key zero page

| Addr      | Use                  |
|-----------|----------------------|
| \$02      | Status A             |
| \$03      | Status B             |
| \$00      | Mode                 |
| \$0F/\$10 | Program step pointer |
| \$0200    | Program RAM base     |
| \$0400    | Music RAM            |
| \$0500    | Speech RAM           |

### 9.3 Motion opcodes

| Hex | Disp | Action          |
|-----|------|-----------------|
| 00  | L    | Left            |
| 01  | F    | Forward         |
| 02  | b    | Reverse         |
| 03  | r    | Right           |
| 04  | Uu   | Wrist up        |
| 05  | Ud   | Wrist down      |
| 06  | Au   | Arms up         |
| 07  | Ad   | Arms down       |
| 08  | Cr   | Claw rotate     |
| 09  | Cc   | Claw open/close |
| 0A  | HL   | Lamp            |
| 0B  | init | Home (00)       |
| 0C  | d    | Delay (sec)     |

### 9.4 Speech / music

| Hex | Disp | Action         |
|-----|------|----------------|
| 0E  | S    | Speech ROM     |
| 0F  | SS   | Speech shift   |
| 10  | PS   | Program speech |
| 81  | PLAY | Play tune      |

| Hex | Disp | Action       |
|-----|------|--------------|
| 82  | SPEE | Speak ROM    |
| 83  | SS   | Speak RAM    |
| 84  | CLr  | Clear phrase |
| FE  | beg  | Marker       |
| FF  | End  | End (FF FF)  |

## 9.5 Cartridge header (\$A000)

| Off                       | Content           |
|---------------------------|-------------------|
| +0                        | Entry vector      |
| +2                        | (c) 1985 CBS Toys |
| +\$13                     | Bootstrap         |
| +\$35                     | Program           |
| +81  <i>Phrases</i>   +BB | Music             |

## 9.6 Common sequences

0C nn    delay nn seconds  
 0B 00    home  
 0A 01    lamp on  
 0A 00    lamp off  
 FF FF    end program

## 9.7 Tools

```
python3 tools/maxx_rom.py validate FILE.532
python3 tools/maxx_rom.py template FILE.532
```

## 10 Schematic Diagram Index

The Commodore 64 Programmer's Reference included a fold-out schematic. This index points to equivalent materials in the Maxx Steele archive.

### 10.1 Main logic board

| Asset                       | Path                                                                                                              | Status                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Raster schematic (enhanced) | Mainboard/Schematic/MaxxSteele_Schematic_enh-v1.1.png<br>(Mainboard/Schematic/Maxx_Steele_Schematic_enh-v1.1.png) | Available                          |
| SVG (v1.1)                  | Mainboard/Schematic/v1.1/MaxxSteele_Schematic.svg<br>(Mainboard/Schematic/v1.1/Maxx_Steele_Schematic.svg)         | Available                          |
| KiCad project               | Mainboard/KiCAD/<br>(Mainboard/KiCAD/)                                                                            | Planned — digitization in progress |

6502 CPU, RAM (M6116), cartridge interface, speech and motor glue.

### 10.2 Transmitter (remote)

| Asset                 | Path                                                                                 | Status    |
|-----------------------|--------------------------------------------------------------------------------------|-----------|
| KiCad schematic + PCB | Transmitter/KiCAD/Transmitter-27MHz.pro<br>(Transmitter/KiCAD/Transmitter-27MHz.pro) | Active    |
| RE notes              | Transmitter/ReverseEngineering/                                                      | Available |
| BOM                   | Transmitter/transmitter-Available<br>bom.md<br>(Transmitter/transmitter-<br>bom.md)  | Available |

COP411L (U1), 455 kHz clock, 27 MHz OOK stage.

### 10.3 Receiver (robot RF)

| Asset           | Path                                                                                           | Status             |
|-----------------|------------------------------------------------------------------------------------------------|--------------------|
| KiCad schematic | Receiver/KiCAD/Receiver-Schematic only<br>27MHz.pro<br>(Receiver/KiCAD/Receiver-<br>27MHz.pro) |                    |
| PCB layout      | —                                                                                              | Not yet in archive |

### 10.4 Cartridge

| Asset          | Path                                           | Status                |
|----------------|------------------------------------------------|-----------------------|
| Schematic scan | Cartridge/Schematic/<br>(Cartridge/Schematic/) | <b>Empty</b> — needed |
| KiCad          | Cartridge/KiCAD/<br>(Cartridge/KiCAD/)         | Planned               |

## 10.5 Face / speech / display

| Asset      | Path                         | Status                             |
|------------|------------------------------|------------------------------------|
| KiCad      | Face/KiCAD/<br>(Face/KiCAD/) | Planned                            |
| Datasheets | DataSheets/<br>(DataSheets/) | LC3100, LC8100, ET9420,<br>COP41xL |

## 10.6 Power module

| Asset | Path                        | Status  |
|-------|-----------------------------|---------|
| KiCad | Power/KiCAD/ (Power/KiCAD/) | Planned |

## 10.7 Paddle mirror accessory

| Asset              | Path                                             | Status                     |
|--------------------|--------------------------------------------------|----------------------------|
| Photo<br>Schematic | PaddleMirror/Photos/ (PaddleMirror/Photos/)<br>— | Available<br><b>Needed</b> |

## 10.8 Chip cross-reference

Indexed datasheets with refdes where known: DataSheets/README.md (DataSheets/README.md).



# ABOUT THE MAXX STEELE... ROBO FORCE TECHNICAL MANUAL...

---

Action figure compatibility... Advanced Robotic Speech and Sound... and high tech battle capabilities make the Maxx Steele Robot Force the most advanced robotic toy in its class for home, play, and educational use.

The MAXX STEELE ROBO FORCE TECHNICAL MANUAL tells you everything you need to know about your Maxx Steele robot. The perfect companion to your Robot Force Commander's Guide, this manual presents detailed information on everything from laser blasters to advanced machine language techniques. This book is a must for everyone from the beginner to the advanced robotic engineer.

For the beginner, the most complicated topics are explained with many sample programs and an easy-to-read wiring style. For the advanced robotic engineer, this book has been subjected to heavy pre-testing with your needs in mind. And it's designed so that you can easily get the most out of your Robot Force capabilities.  
engineer.

For the beginner, the most complicated topics are explained with many sample programs and an easy-to-read wiring style. For the advanced robotic engineer, this book has been subjected to heavy pre-testing with your needs in mind. And it's designed so that you can easily get the most out of your Robot Force capabilities.



**maxx steele**  
**ROBO FORCE**

Maxx Steele Robot Force, Inc. — Robot Systems Division.  
1301 Western Avenue, Cincinnati, Ohio 45203